diffmrc Manual

Reference, methods, and verification records

Version 0.1

The reference document for the diffmrc package: installation and configuration, the API surface, protocol details, the design of the differentiable chain and its second derivatives, the verification records, benchmark results, and the development architecture. The companion *Tutorial* covers first use.

Contents

I User reference **4**

1 Overview and package layout **5**
1.1 Design principles . . . 5
1.2 Package layout . . . 5
1.3 How the documents are organized . . . 6

2 Installation and configuration **7**
2.1 Supported environment . . . 7
2.2 Runtime configuration . . . 7
2.3 Verifying the installation . . . 7
2.4 Optional: running the figures . . . 8
2.5 Known environment friction . . . 8

3 API reference **9**
3.1 core: the numerical kernel . . . 9
3.2 analysis: NumPy only . . . 9
3.3 policy: selection rules . . . 10
3.4 driver: the differentiable pipelines . . . 10
3.5 cli . . . 10

4 Protocols and options **11**
4.1 Orbital-point convergence . . . 11
4.2 State averaging . . . 11
4.3 Active-space windows . . . 12
4.4 The e2_scale switch . . . 12
4.5 Differentiation settings . . . 12

II Methods and records **13**

5 The differentiable chain **14**
5.1 Parameterization . . . 14
5.2 The two factory families, and why . . . 14
5.3 Inside the chain . . . 14
5.4 What is verified, and against what . . . 15

6 Second derivatives **16**
6.1 The nuclear Hessian by implicit differentiation . . . 16
6.2 The second-order stages never materialize the Hessian . . . 16
6.3 Degenerate pairs and the derivative basis . . . 16
6.4 From the Hessian to frequencies, and one cautionary record . . . 17

7 Verification records **18**
7.1 Energy chain . . . 18
7.2 First derivatives . . . 18
7.3 Second derivatives . . . 19
7.4 External-data cross-checks . . . 19
7.5 The layer contracts . . . 19

8 Benchmarks **20**
8.1 Ground-state line (M5-A) . . . 20
8.2 Excited-state line (QUEST subset) . . . 20
8.3 Selection-protocol cross-check (DVS-APC) . . . 21
8.4 What is not benchmarked . . . 21

9 Known limitations **22**
9.1 Method-family limits . . . 22
9.2 Implementation limits . . . 22
9.2.1 Degenerate pairs at a stationary point . . . 23
9.3 Protocol limits . . . 23

III Development **24**

10 Architecture and contracts **25**
10.1 The six layers . . . 25
10.2 Contracts are tests . . . 25
10.3 How the test suite is organized . . . 25
10.4 Development workflow . . . 26

11 Reproducing the paper **27**
11.1 Environment . . . 27
11.2 The paper's figures and tables . . . 27
11.3 The later result lines . . . 27
11.4 Building this documentation . . . 28

Part I

User reference

Chapter 1

Overview and package layout

1.1 Design principles

Three commitments shape the package, and most of its structure follows from them.

The first is that derivatives are never written by hand. Every nuclear gradient and second derivative is obtained by differentiating the energy code path with JAX. This is what makes the CASSCF, state-averaged CASSCF, and NEVPT2 layers share one implementation of the physics: a correction to the energy expression propagates to all of its derivatives without a separate derivation.

The second is that the package is layered, and the layer boundaries are tested. The kernel cannot import the layers above it; the analysis layer cannot import the kernel at all. The contract is enforced in the test suite (see Chapter 10), so a violation is a failing test rather than a matter of review discipline.

The third is that convergence is reported, not assumed. Optimizers return the gradient norm at the point they stopped at, and the package distinguishes "the optimizer stopped improving" from "the point is stationary." Callers are expected to carry that distinction into whatever they claim about a result.

1.2 Package layout

```
src/diffmrc/
  core/       numerical kernel: rotations, CASCI, CASSCF, NEVPT2,
              stationary points, the frozen source of the chain
  analysis/   post-processing that reads converged objects: entropies,
              mutual information, spin and state identification,
              orbital identity and frontier windows; NumPy only
  policy/     selection rules over analysis output: composition,
              entropy-based and ranked orbital selection, verification
  driver/     composition layer: assembles the kernel into the
              differentiable SCF -> CASSCF -> NEVPT2 pipelines,
              orbital-point convergence, frozen-geometry factories
  report/     result objects and rendering
  cli/        the command-line entry point
```

The dependency direction runs downward only: `core` at the bottom, `cli` at the top. A user script may import any layer; the layers themselves may not import upward. This is what allows the analysis layer to be tested and used with nothing but NumPy installed.

1.3 How the documents are organized

This manual is the reference. Part I covers installation, the API surface, and the protocol details that the code reports but does not enforce. Part II carries the methods: the design of the differentiable chain, the second-derivative construction, the verification records against independent implementations, the benchmark results, and the consolidated limitation list. Part III is for development: layer contracts, the test architecture, and the reproduction steps for the paper.

The companion *Tutorial* is the on-ramp: installation, a first CASSCF gradient, state-averaged workflows, active-space selection, and a full benchmark walkthrough, each with runnable code.

Chapter 2

Installation and configuration

2.1 Supported environment

The package is developed and verified on Linux and Windows with CPython 3.11, on CPU. The Tutorial (Chapter 2) gives the minimal recipe; this chapter records the constraints behind it.

Dependency	Constraint	Why it is pinned
Python	≥ 3.10	`int.bit_count` in the determinant tools
numpy	≥ 1.24	analysis layer, reference arithmetic
scipy	≥ 1.10	L-BFGS-B and trust-region stages
pyscfad	$\geq 0.3.3$	provides the traceable SCF and integral path
pyscf	≥ 2.5	reference energies and comparisons
jax	≥ 0.4	array layer; the exact pin is pyscfad's
matplotlib	any recent	figure sources only

`pyscfad` pins a specific JAX version, which is why the package should live in its own environment. Mixing it into an environment with a different JAX pin produces failures that look like numerical bugs and are not.

2.2 Runtime configuration

Three settings are load-bearing.

`jax_enable_x64 = True`. The package is validated in double precision. With the default single precision the errors are not loud: gradients lose several digits and some convergence gates become unreachable, while energies look plausible.

`JAX_PLATFORMS = cpu`. The CPU platform is the verified configuration. On a machine with a CUDA device, leaving the platform unset selects the GPU, where the solver path used by the orbital canonicalization raises an error unrelated to the chemistry. Set the variable before importing JAX; setting it afterwards has no effect.

Thread environment variables. The driver assumes it owns the thread budget it was given. Setting the five BLAS thread variables keeps multiple concurrent calculations from oversubscribing a node; their absence does not corrupt results, it makes timing meaningless and slows everything.

2.3 Verifying the installation

```
pytest tests/ -v
lint-imports        # or: pytest tests/test_layer_contracts.py
```

The suite is the primary artifact to look at: it contains the first-order and second-order derivative checks against finite differences, the cross-validation against PySCF, and the layer contracts. A passing suite on your machine is the intended proof that the environment is sound.

2.4 Optional: running the figures

The figure sources under `figures/` import matplotlib and read JSON results produced by the scripts; they require no additional configuration. They are not part of the package tests.

2.5 Known environment friction

- The first evaluation of a freshly built node of the chain triggers XLA compilation that can take minutes. This is expected; a stopwatch on a single evaluation will mislead.
- pyscfad's `Mole` objects cannot cross a `jax.jit` boundary. The driver layer offers frozen-geometry factories for jitted contexts; the factory docstrings state which is which.
- On shared nodes, leave headroom for the owner of the other processes. The memory profile of this package is dominated by compilation- and evaluation-time buffers of the differentiated chain, not by the integrals, and it grows steeply with the number of rotation parameters.

Chapter 3

API reference

This chapter lists the entry points a user is expected to reach for, layer by layer. It is a map, not a transcription: the authoritative signature of every function is its docstring in the source, and the module-level `__all__` lists are the canonical inventories. Entries marked *stable* have been exercised by the verification records in Chapter 7; the rest follow the same conventions but have lighter coverage.

3.1 core: the numerical kernel

Entry point	Purpose
`rotation.block_rotation(t, nmo, pairs)`	rotation matrix from the parameter vector
`orblib.orbital_pairs(ncore, nact, nmo)`	the non-redundant rotation pairs
`casci.cs_casci(...)`	CASCI energies; state-averaged branch with `nroots`, `return`
`optimizer.optimize_theta_scipy(...)`	L-BFGS-B stage
`optimizer.optimize_theta_hand(...)`	the fallback when scipy raises
`grad.make_energy_fn / make_grad_fn`	energy and nuclear-gradient callables
`grad.solve_theta_star(...)`	the two-stage orbital optimization
`grad.nuclear_gradient(mol, grad_fn)`	the differentiated gradient
`stationary.refine_stationary_point(...)`	damped-Newton refinement (Newton–CG step)
`stationary.curvature_report(H)`	the saddle-point check
`stationary.StopReason / StationaryPoint`	convergence verdict types
`nevpt2.impl.cas_dets_for2(...)`	determinant bitmaps of the active space
`nevpt2.impl.canonicalize_jax(...)`	orbital canonicalization on the differentiated path
`nevpt2.impl.nevpt2_e2_full2(...)`	the frozen NEVPT2 contraction

The kernel is frozen: later layers build on it and corrections to it go through the verification records, not through ad-hoc edits.

3.2 analysis: NumPy only

Entry point	Purpose
`states.state_groups / select_groups`	roots to physical states; state selection
`states.spin_square / det_abs_sz`	exact $\langle S^2 \rangle$ and $\lvert M \rvert$ of determinants
`orbitals.frontier_report / classify_orbital`	frontier identities; window suggestion
`orbitals.heavy_atoms / p_axis_profile`	atom keys; per-axis p profiles
`ci.occupations_from_ci / entropy_from_ci`	occupations and entropies from a CI vector
`ci.pair_density / mutual_information`	two-orbital reductions and their correlations
`rdm.entropy_from_rdms`	the density-matrix route to the same quantities
`diagnostics.is_single_reference / zs1`	multireference diagnostics
`plateau.detect_plateaus`	entropic plateaus of an orbital set

This layer imports nothing from the kernel; it operates on the determinant-bitmap convention and on arrays, and its tests run with NumPy alone.

3.3 policy: selection rules

The policy layer turns analysis output into selections: composition-based rules, entropy-based ordering, the ranked orbital selection with its singly-occupied and virtual-removal options, and the verification helpers. The module names are the inventory (`policy.composition`, `policy.entropy`, `policy.ranking`, `policy.rules`, and their siblings); the docstrings state the conventions each rule follows.

3.4 driver: the differentiable pipelines

Entry point	Purpose
`nevpt2.make_nevpt2_joint_fn(...)`	(mol, θ) SCF→CASSCF→NEVPT2, geometry-diffe
`nevpt2.make_nevpt2_theta_fn(...)`	frozen geometry, θ only; jit-safe
`nevpt2.make_nevpt2_sa_fn(...)`	state-averaged joint factory
`nevpt2.make_nevpt2_sa_theta_fn(...)`	state-averaged, frozen geometry, jit-safe
`nevpt2.sa_cas_solution(...)`	state-averaged CAS solution and CI vectors
`nevpt2.sa_nevpt2_state_table(out, dets)`	grouped physical-state table
`nevpt2.sa_vertical_excitations(states)`	per-state vertical excitation energies
`orbitals.converge_orbital_point(E_theta, npar)`	two-stage convergence with refinement and polish
`orbitals.hessp_of(E_theta)`	Hessian-vector product for the trust-region stage

The two factory families exist because of a pyscfad constraint: the `Mole`-based factories cannot be traced inside `jax.jit`, and the theta-only factories are built for exactly that context. When in doubt about which to call, ask whether the geometry is fixed; if it is, use the theta-only factory.

3.5 cli

`python -m diffmrc.cli` runs an end-to-end calculation from a geometry file; `--help` lists the options. The command-line entry is the least settled surface of the package.

Chapter 4

Protocols and options

The package reports convergence and protocol evidence; it does not enforce it. This chapter collects the settings that decide whether a number is trustworthy, with the values the benchmark records were produced under. Where a default can be changed by a caller, the change belongs next to the number it produced, because several of the gates below are reachable at more than one tolerance and the results differ at the margins.

4.1 Orbital-point convergence

The two-stage protocol separates two different failure modes: the optimizer that can no longer lower the energy, and a point that is not stationary. The stages and their defaults:

Setting	Value	Meaning
`optimize_tol`	10^{-6}	L-BFGS-B target; loose on purpose
`refine_target`	10^{-11}	the Newton target, and the verdict
`max_steps`	2000	L-BFGS-B iteration cap
`refine_max_iter`	80	damped-Newton iteration cap
`_CG_tol` / `_CG_maxiter`	10^{-6} / 40	Newton–CG budget for the step solve

The verdict is the gradient norm at the point the chain stopped at, classified by the stop reason:

`StopReason`	Reading
`CONVERGED`	gradient below the refine target
`STALLED_GRADIENT`	a step no longer reduced the gradient
`GUARD_STOPPED`	no damped step lowered the energy
`ITERATION_CAP`	budget exhausted; re-check before claiming convergence

Two details of the Newton stage are load-bearing and easy to lose when reimplementing. The Hessian enters only through Hessian-vector products — the explicit `jax.hessian` broadcasts every intermediate over n_{par}^2 tangent pairs and was killed by a 402 GB allocation at $n_{\mathrm{par}} \approx 500$ — and the downhill acceptance test is not optional: an unguarded refinement moved an energy from -76.07 to $+1.04$ Ha in this project's history and printed no error.

4.2 State averaging

The state-averaged factories take `nroots` and `state_weights`; the weights are normalized and must have one entry per root. The measured effect of between-group equal weights (the second pass) on the reported excitation energies stayed within a few tenths of an eV for the benchmark

systems, while the second pass usually tightens the final orbital gradient by orders of magnitude. The benchmark records quote which pass each number came from.

Two root-table facts belong with every state-averaged number. The eigensolver returns roots, so a triplet travels as three degenerate roots; and a per-component NEVPT2 correction can be undefined (`nan`) for a pure $|M| = 1$ component, in which case the group's representative is the lowest component with a finite value. `sa_nevpt2_state_table` performs both bookkeeping and returns the evidence.

4.3 Active-space windows

The frontier-window rules are stated in the Tutorial (its chapter on choosing an active space). The part that belongs to a protocol table is the bookkeeping: the electron count follows the window,

$$n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2\, n_{\mathrm{core}}, \tag{4.1}$$

and a window that widens to three orbitals for the linear two-pi and amide patterns therefore carries four active electrons, not two. A hard-coded two here produces a wrong ground state that the energy alone will not flag.

4.4 The e2_scale switch

Both NEVPT2 factories accept `e2_scale`. With `e2_scale = 0` the perturbation correction is switched off and the call reduces to the CASSCF energy of the same chain — the identity used by the test suite to pin the joint and state-averaged factories against the validated single-root path. With the default value of 1.0 the correction is included. There is no other intermediate value with a physical meaning; the switch exists for verification.

4.5 Differentiation settings

The theta-only factories are safe inside `jax.jit`; the `Mole`-based ones are not, and the failure mode is a tracing error at compile time rather than a wrong number. The second-order paths (the refinement, the trust-region polish) differentiate the gradient with forward-mode `jvp` calls; a caller wrapping the whole convergence in `jit` should expect the compile of those nodes to dominate the first evaluation.

Part II

Methods and records

Chapter 5

The differentiable chain

This chapter describes how the package makes a multireference energy differentiable, and records the numerical facts that shaped the design. It is the software-side companion to the methods paper; the derivations live there, the implementation constraints live here.

5.1 Parameterization

Orbital variation enters through a rotation parameter vector θ: the occupied–virtual mixing is generated by the non-redundant pair list of `orblib.orbital_pairs`, and `rotation.block_rotation` turns θ into the rotation matrix applied to the reference orbitals. Three pair families are included: core–active, core–virtual, and active–virtual, for every orbital outside the active set.

The active–active rotations are omitted deliberately: they are redundant against the CI step, which is complete inside the active space. The core–virtual family is *not* omittable. A parameterization that drops it stops at a point that is not stationary; the defect was measured at the level of a few hundredths of a kcal/mol on a test system — small, systematic, and invisible without an independent reference. The pair list is the contract; do not prune it for speed.

5.2 The two factory families, and why

Every differentiable entry point comes in two forms.

The *joint* factories (`make_nevpt2_joint_fn` and its state-averaged sibling) take the molecule and θ and run the reference SCF inside the call. They are the geometry-differentiable path: the nuclear coordinates enter through the integrals, and JAX differentiates the whole composition.

The *theta-only* factories (`make_nevpt2_theta_fn`, `make_nevpt2_sa_theta_fn`) compute the reference SCF and the integrals once, at construction, and return a callable of θ alone. They exist for one hard reason: this build of pyscfad cannot trace its SCF inside `jax.jit`, and the orbital optimizer wraps its objective in `jit`. A joint factory called from inside the optimizer raises a tracing error; the theta-only factory is the supported path there. The two families funnel through one shared component builder, so they cannot drift apart numerically — the test suite pins them against each other at fixed geometry.

5.3 Inside the chain

A single evaluation composes four numerically delicate pieces.

The MO integral transform runs in four contraction steps. The four-operand form of the two-electron transform is numerically weaker on XLA at large sizes, and the discrepancy is not cosmetic: on an 96^4 integral array the one-shot form showed absolute errors around 10^{-6}, which the near-degenerate gaps of a linear molecule's generalized Fock matrix amplify into a completely different eigenvector basis inside the degenerate subspace. The energy is insensitive

to that; the theta gradient differed by eight percent between the two contraction orders. The stepwise form is therefore the implementation, not an optimization.

Canonicalization is differentiable and state-aware. The NEVPT2 denominators are defined on the canonical (natural-orbital, sub-Fock-diagonalized) set, so the contraction runs on canonicalized orbitals produced inside the differentiated path. The state-averaged variant rotates with the average density of the state-averaged solution; at one root it reproduces the ground-state canonicalization bit for bit, which is how the two are pinned against each other.

Per-state energies are read out of the eigensolver's spectrum. The state-averaged solver returns the mean of the lowest roots; the per-state energies are recovered from the running means by the sliding-mean inversion $E_k = k\,M_k - (k-1)\,M_{k-1}$. The number of roots is silently clamped by the size of the determinant space, so the inversion is performed over the effective root count, not the requested one.

Degeneracies are handled at the decomposition level. An eigensolve with degenerate eigenvalues has no unique derivative basis; the kernels use degeneracy-safe decompositions throughout, and the choice of cluster tolerance is a protocol decision with its own limitation entry (Chapter 9).

5.4 What is verified, and against what

The chain's verification record is deliberately built on invariance and cross-implementation checks rather than on agreement with any single reference: the CASSCF gradient is validated against invariance conditions and against an independent derivative-free route; the NEVPT2 energies against an independent implementation; the state-averaged addition against the single-root path at one root. Chapter 7 lists the records with their measured deviations.

Chapter 6

Second derivatives

The package differentiates its energy chain twice, so that harmonic frequencies and curvature checks come out of the same code path that produced the gradients. This chapter records the construction and the numerical facts that forced their design.

6.1 The nuclear Hessian by implicit differentiation

Differentiating through a converged orbital optimization naively — differentiating the unrolled iteration — scales with the number of iterations and inherits every numerical accident of the solver. The production route instead treats the optimized orbital parameters as an implicit function of the geometry, $\theta^*(R)$, and assembles the second derivative from the partial derivatives of the energy at the converged point plus a response term built from the orbital Hessian.

Two implementation facts came out of the verification work. The response solve must use a pseudoinverse rather than a plain solve: the orbital Hessian at a converged point is singular in the directions the optimizer is free to ignore, and the plain solve either fails or returns noise. And the implicitly assembled Hessian must be pinned against the alternative construction: the two routes agree to 1.6×10^{-11} on the validation systems, with the implicit route costing about a third of the unrolled one.

6.2 The second-order stages never materialize the Hessian

The orbital refinement (damped Newton) and the trust-region polish need the Hessian only through its action on vectors. The earlier implementation built it explicitly with `jax.hessian`, which is forward-over-forward: every intermediate of the chain is broadcast over the square of the parameter count. At five hundred parameters that construction requested a single 402 GB allocation and died. Both stages now receive Hessian-vector products — a forward-mode derivative of the gradient — which carry one tangent per reverse pass: the refinement solves its Newton step by conjugate gradients on those products, and the trust-region stage consumes them through the optimizer's `hessp` interface. The budgets are deliberately shallow (tolerance 10^{-6}, forty iterations): an inexact Newton step feeds a damping ladder and a downhill acceptance test that correct it. The deep solver cost hours of product evaluations per molecule, and the shallow budget passes the same refinement regressions.

6.3 Degenerate pairs and the derivative basis

A symmetry-degenerate pair of orbitals has no canonical derivative basis: the eigenvectors inside the degenerate subspace are arbitrary, so a naive derivative of a decomposition returns a quantity that depends on the solver's internal ordering. For a linear molecule this is not an edge case — the π pairs are exactly degenerate, and a gradient assembled through an unguarded decomposition

returns NaN. The kernels use degeneracy-safe eigensolvers throughout the differentiated path, and the second-order routes avoid differentiating decompositions where the response construction allows it.

6.4 From the Hessian to frequencies, and one cautionary record

Harmonic frequencies follow from the mass-weighted Hessian by the standard normal-mode analysis, using PySCF's driver on a Hessian this package assembles. The conversion has a layout trap that cost this project a wrong number: the Hessian must be handed over in the (atom, coordinate) block layout the thermodynamics module expects, not as a plain reshape. The wrong layout comes out plausible and silent — for CO at 6-31G it produced 1501.8 cm^{-1} against the correct 2293.5 cm^{-1}.

The frequency route is anchored against an independent value: the CO molecule at 6-31G gives 2294.2 and 2293.5 cm^{-1} from the two assembly routes, against an external anchor at 2293.5 cm^{-1}. When a validation target sits near zero, do not reach for finite differences: this project measured finite-difference errors of over one hundredfold on a near-zero mixed derivative, growing as the step shrank. Use an invariance or an analytic cross-check instead.

Chapter 7

Verification records

Every claim this package makes about itself rests on a recorded comparison. This chapter lists the records by the question they answer, with the measured deviation and the test or script that holds it. Where a check is an equality that must hold by construction, the deviation is quoted at machine precision; where it is a comparison against an independent implementation or an external reference, the deviation is quoted with its protocol.

7.1 Energy chain

Question	Measured	Where
CASCI/CASSCF against PySCF, same geometry, basis and window	10^{-12} to 10^{-10} Ha (LiH, HF, NH_3)	`t305`; driver kernels
NEVPT2 correction against an independent implementation	4×10^{-7} on the large-core test after the block-loop fix	the frozen-contraction tests
State-averaged solution at one root against the ground-state path	bit-for-bit (10^{-12})	`tests/driver/test_sa_pipeline.py`
Joint and theta-only factories at fixed geometry	10^{-10} Ha	the same suite

7.2 First derivatives

The nuclear gradient is validated by invariance rather than by agreement with a single reference, and then by a regression against an independent derivative-free route:

- the CASSCF gradient is invariant under coordinate transformations at the 10^{-14} level, and reproduces the finite-difference reference to its own noise on smooth systems;
- the regression against the analytic reference reproduces the reference to the level at which both are converged; departures appear at points where the reference geometry was not stationary, which is a property of the comparison, not of the gradient;
- a near-zero mixed derivative is a known trap for finite differences: measured errors of over one hundredfold on a derivative whose value sits near zero, growing as the difference step shrinks. Such checks use invariances instead.

7.3 Second derivatives

Question	Measured	Where
Implicit route against the unrolled construction	1.6×10^{-11}	`t154`; the implicit-Hessian tests
Harmonic frequency against an external anchor (CO, 6-31G)	2294.2 and 2293.5 cm^{-1} from the two routes, against 2293.5 cm^{-1}	frequency tests
Hessian-vector product against the explicit Hessian	10^{-10} on a non-quadratic fixture	`tests/driver/test_orbitals.py`
Damped-Newton step solver (CG) against a dense solve	10^{-8} on SPD fixtures	`tests/core/test_stationary.py`

7.4 External-data cross-checks

Two independent data lines were used to check layers that have no internal ground truth:

- **A published selection-protocol data set** (DVS-APC, Zenodo record, CC-BY-4.0): the energy layer reproduces three reference state energies to 10^{-11} Ha; a port of the published entropy-based selection formula, evaluated on all 116 orbitals of every record, agrees with the published values to exactly zero difference; the singly-occupied rule of the protocol is confirmed on the full set rather than on a sample.
- **An external benchmark database** (QUEST): the state-averaged excitation pipeline is checked over fourteen states of seven molecules; the records and the family split are in Chapter 8 and the Tutorial.

7.5 The layer contracts

The six-layer architecture is itself under test: `tests/test_layer_contracts.py` (or the `lint-imports` entry point) parses the import graph and fails on any upward import. The check is structural, not conventional: an import performed lazily inside a function body is still reported, because the contract is parsed from the syntax tree.

Chapter 8

Benchmarks

Two benchmark lines exercise the package end to end. The first is a ground-state line against direct PySCF runs at matched settings; the second is the excited-state line against the QUEST reference set, whose protocol and reading are walked through in the Tutorial (Chapter on the benchmark walkthrough). This chapter records both with their conventions, so that a quoted number can be traced back to its run.

8.1 Ground-state line (M5-A)

Three closed-shell molecules at matched geometry, basis, and active window, chain against PySCF:

System	Deviation (Ha)	Note
LiH	-3.5×10^{-12}	active window [4,5], 6-31G
HF	-3.6×10^{-13}	active window [1,2], 6-31G
NH_3	-1.9×10^{-10}	active window [1,2], 6-31G

The agreement is at the level where the two codes' convergence tolerances meet. The chain's end-to-end entry reports the energy and the stationarity diagnostics; nuclear gradients on this line come from the lower-level interface and are covered by the first-derivative records of Chapter 7.

8.2 Excited-state line (QUEST subset)

Fourteen vertical excitation energies over seven molecules, 6-31G, the frontier-window protocol with its corrections, against the QUEST database's best estimates and its own SC-NEVPT2 column. The full table sits in the Tutorial; the summary by family:

Family	MAD vs best estimate	MAD vs SC-NEVPT2
$n \to \pi^*$ (8 states)	0.31 eV	0.36 eV
$\pi \to \pi^*$ (6 states)	1.14 eV	1.08 eV

Two protocol facts belong to these numbers. The basis is 6-31G while the reference set's values are near the complete-basis limit, so part of every deviation is basis, not method; and the window corrections of the Tutorial's active-space chapter move individual entries by up to a factor of two in deviation before the family statistics are computed. The residual $\pi\pi^*$ deviation — uniform in sign across five chemically different systems — is recorded as a capability boundary of the minimal-space protocol at this basis, not as a molecule-specific error.

8.3 Selection-protocol cross-check (DVS-APC)

The entropy-based active-space selection layer was checked against a published automated-selection data set on its own terms: a port of the published formula evaluated on every orbital of every record agrees with the published entropy values to exactly zero difference, the energy layer reproduces reference state energies to 10^{-11} Ha, and the protocol's singly-occupied rule is confirmed on the full data set. The cross-check also fixed a defect this package had carried: the published protocol includes a virtual-removal correction of order N that the port initially lacked; with the correction in place, the agreement statistic reaches a mean of 0.63 over the full selection set on the primary data variant and 0.59 on the alternative, roughly six times a random baseline. The per-molecule statistics, including the cases where the two variants disagree, are in the stored report.

8.4 What is not benchmarked

Large-active-space behaviour, GPU execution, spin–orbit coupled targets, and relativistic Hamiltonians have no benchmark here because they are outside the present scope (Chapter 9). A number produced by this package on those targets has no verification record behind it.

Chapter 9

Known limitations

This is the consolidated list. Each entry states the limitation, the measurement behind it where one exists, and the check a user can run. The Tutorial has the same material in shorter form; the entries here carry the numbers.

9.1 Method-family limits

Contracted NEVPT2 accuracy. The perturbation layer inherits the standard contracted-NEVPT2 approximations; the reference set's own MAE for the method is the accuracy band claimed here, not a target this package improves on.

Valence–Rydberg mixing. $\pi\pi^*$ states near Rydberg states move by up to an electron volt when diffuse functions enter the basis. Measured on ethylene at a fixed geometry: the singlet $\pi\pi^*$ vertical excitation moves from 9.27 to 8.09 eV between 6-31G and 6-31+G* in the same three-orbital window, while the triplet moves the opposite way (4.83 to 5.68 eV); the singlet–triplet gap, which the reference fixes near 3.4 eV, collapses from 4.44 to 2.41 eV. A gap collapse in the wrong direction is the signature of a mixed-in state, not a converged one. Check: re-run with and without diffuse functions.

Minimal-window limits for multiconfigurational $\pi\pi^*$ states. The benchmark record shows a uniform positive deviation of the $\pi\pi^*$ family (+0.6 to +1.5 eV) that survives every window correction tried. Check: widen the window by one occupied orbital; a deviation that shrinks is window-limited.

No spin–orbit coupling; scalar-relativistic with ECPs only. Fine structure is out of scope.

9.2 Implementation limits

Dense diagonalization. The CI step materializes and diagonalizes the full determinant space; the practical ceiling is a few thousand determinants per root calculation. A differentiable iterative eigensolver is future work.

Second-derivative cost. The implicit route requires the orbital Hessian in memory and does not exploit sparsity; the unrolled route scales with the iteration count. Neither is competitive with analytic second derivatives of single-reference methods.

Memory and compile profiles of the differentiated chain. Intermediates are broadcast over the rotation-parameter count. The measured record that shaped the current design: an explicit Hessian at about five hundred parameters requested a 402 GB allocation in a single request; the Hessian-vector-product stages that replaced it carry one tangent per pass, and their first evaluation still spends minutes in XLA compilation at that size. Check: watch resident memory during the first optimization step, and leave headroom on shared machines.

9.2.1 Degenerate pairs at a stationary point

The eigensolver's cluster tolerance must separate two scales: the floating-point noise that splits symmetry-degenerate pairs at a rotated stationary point (measured between 10^{-8} and 1.2×10^{-5} on the systems in this work) and the physical gaps of the same spectrum (1.9×10^{-2} and above). A tolerance below the noise exposes the first and second derivatives to a bare inverse gap; one above the physical gaps merges distinct states. The window measured here spans 10^{-5} to 10^{-2}, and there is no admissible value when a physical gap falls inside the noise band — the fallback for that case is the decomposition-free Lanczos kernel. Check: if a linear or otherwise degenerate system returns unstable derivatives, this tolerance is the first knob to inspect.

9.3 Protocol limits

Multiple stationary points. The orbital energy is non-convex; different starting points reach different stationary points, and the selection is not stable even at fixed settings — water at 6-31G has two minima 13 kcal/mol apart, and the reference-orbital start reaches a saddle. Report the stationarity certificate and the stationary-point spread with every multireference number.

NEVPT2 convergence sensitivity. The correction depends on the orbitals more sensitively than the energy; energy-based gates are insufficient when the target is the correction. Measured: orbital stalls that move the correction by several kcal/mol across runs whose energies agree to the microhartree. Use the stability of the correction across independent runs together with a tightened orbital-gradient gate.

Window rules and bookkeeping. The frontier audit covers the patterns met so far; the electron count follows the window ($n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2n_{\mathrm{core}}$) and a hard-coded count produces a wrong ground state that the energy alone does not flag.

Part III

Development

Chapter 10

Architecture and contracts

10.1 The six layers

Layer	Responsibility
`core`	the numerical kernel: rotations, CASCI, CASSCF, NEVPT2, stationary points. Frozen: correctio
`analysis`	post-processing of converged objects; NumPy only; no kernel import.
`policy`	selection rules over analysis output.
`driver`	composition: the differentiable SCF–CASSCF–NEVPT2 pipelines, orbital-point convergence, froz
`report`	result objects and rendering.
`cli`	the command-line entry point.

Dependencies point downward only. A user script may import any layer; the layers themselves may not import upward. The most consequential of these rules is the analysis layer's isolation: it must run with nothing but NumPy installed, which is what keeps its diagnostics usable inside any environment.

10.2 Contracts are tests

The import direction is enforced mechanically, not by review: `tests/test_layer_contracts.py` parses the syntax tree of every module and fails on any upward import. The check has a deliberate property — an import hidden inside a function body is still reported, because the contract is structural. The `lint-imports` configuration in the package metadata encodes the same rules for environments where the import-linter tool is available.

10.3 How the test suite is organized

The suite mirrors the layers: `tests/core/`, `tests/analysis/`, `tests/policy/`, `tests/driver/`, `tests/report/`, plus cross-layer integration tests. Two conventions are worth knowing before adding tests:

- The analysis tests run on synthetic fixtures with hand-built numbers and no chemistry package import; this is what keeps the layer's tests runnable anywhere. The conventions under test (determinant bitmaps, AO labels) are stated in the fixtures themselves.
- The driver tests bind the layer against the kernel: the frozen factories are pinned against the joint ones, the state-averaged solution against the single-root path, and the Hessian-vector product against the explicit Hessian. When a change touches the chain, one of these bindings is the test that should notice.

10.4 Development workflow

Non-trivial additions start as a written plan, are implemented test-first, and land with the verification record that justifies them. Bug fixes follow the same rule: the record is the fix's justification, and the test is the record made executable. The differentiable-chain and second-derivative chapters were developed in that order, and several of their numerical facts are traceable to the record that found them.

Chapter 11

Reproducing the paper

11.1 Environment

Every reproduction script sets its own runtime environment at the top: the five BLAS thread variables and `JAX_PLATFORMS=cpu` before the first JAX import, and `jax_enable_x64` immediately after. A script run outside this convention may produce different timings and, in the single-precision case, wrong numbers; the settings are quoted in Chapter 2.

The scripts write JSON results next to themselves (or into a `results/` directory); the figure sources read those JSON files. Run them from a directory with the package importable, or adjust the path insert at the top of the script.

11.2 The paper's figures and tables

Script	Result
`t157_grad_gold_fix.py`	nuclear-gradient validation; invariance conditions
`t155_grad_regression.py`	gradient regression against an analytic reference
`t150_gold_refine.py`	golden-standard Hessian by re-optimizing at each displaced geometry
`t149_mix_AD_vs_FD.py`	why finite differences fail for the mixed derivative
`t151_unrolled_theta.py`	response term by unrolled iteration
`t152_generalize.py`	unrolled route on CO and N_2
`t154_implicit_hessian.py`	response term by implicit differentiation
`t153_hessian_to_freq.py`	harmonic frequencies from the mass-weighted Hessian
`t159_basis_active_space.py`	frequency dependence on basis set and active space
`t160_collect_freq.py`	frequency summary
`t142_layerA_v2.py`, `t156`, `t158`	lanthanide dissociation energies

11.3 The later result lines

The state-averaged and benchmark lines that postdate the paper's first draft live under the project's `code/` directory:

Script	Result
`t315 / t317 / t321`	the QUEST-subset excitation runs (with `t320` the window audit)
`t316`	the assembled benchmark table (reads the recorded JSON)
`t318`	the large-molecule single-stage protocol
`t306`	the first state-averaged targets, with the two-pass weights
`t300`–`t302`	the external selection-protocol cross-check

Each of these carries its conventions in its docstring: basis, geometry source, window rule, state-weighting pass. A number quoted from one of them without those four facts is a number that will not survive being compared with the others.

11.4 Building this documentation

The Tutorial and the Manual are xelatex documents under `manual/`; compile each twice:

```
cd manual/tutorial && xelatex tutorial.tex && xelatex tutorial.tex
cd ../manual    && xelatex manual.tex    && xelatex manual.tex
```